CHECKIN

מערכת נוכחות ושכר לצוותים

דיווח כניסה ויציאה מהמשרד או מהבית, דוחות לראש הצוות, תיקון שעות מבוקר, אישורי מחלה וחישוב שכר אוטומטי.

🙏 תודות

נפתח בתודה למי שהכול שלו וממנו הכול — **הקדוש ברוך הוא**. ללא סייעתא דשמיא לא הייתי מצליחה לפתח את האתר הזה.

תודה למורה על קורס שלם של לימוד, גם בזמנים לחוצים. תודה רבה על הכול: על היחס, על ההקשבה, על השיתופים הקטנים, ועל האמונה והעקרונות שהמורה החדירה בין השיעורים.

תודה ענקית!

🌐 האתר באוויר

מה	קישור
האתר	https://checkin-client-tml7.onrender.com
קוד הלקוח (React)	https://github.com/Dini167/CheckIn-client
קוד השרת (Node + PostgreSQL)	https://github.com/Dini167/CheckIn-server

השרת רץ במסלול החינמי של Render ו"נרדם" אחרי זמן בלי שימוש — הכניסה הראשונה יכולה לקחת עד דקה.

👤 משתמשי דמו

תפקיד	שם	אימייל	סיסמה
ראש צוות	דנה ישראלי	`dana@example.com`	`Dana-Demo-2026`
עובד	יוסי לוי	`yossi@example.com`	`Worker-Demo-2026`
עובדת	נועה כהן	`noa@example.com`	`Worker-Demo-2026`

- **קוד הצוות:** `TEAM-DEMO1` (צוות פיתוח) — להרשמה של עובד חדש לצוות.

- במערכת יש היסטוריית נוכחות לדוגמה של כחודש, כולל ימים עם חסר בדיווח, בקשת תיקון שממתינה לאישור, יום מחלה וחג.
- כל הנתונים בדויים.

🎯 למה נוצרה המערכת

בעסקים קטנים ובינוניים, דיווח השעות נעשה הרבה פעמים בטבלת אקסל, בכרטיס נייר או בהודעת וואטסאפ למנהל. כשחלק מהעובדים עובדים מהבית, המצב מסתבך עוד יותר: קשה לדעת מי עובד עכשיו, שעות נשכחות, תיקונים נעשים בלי בקרה, ובסוף החודש מישהו צריך לשבת ולחשב שכר ידנית.

CheckIn מרכזת את כל זה במקום אחד: העובד מדווח בלחיצה, ראש הצוות רואה תמונה מלאה בזמן אמת ומאשר חריגים, והשכר מחושב לבד מתוך השעות.

👥 למי המערכת מיועדת

למי	מה הוא מקבל
בעלי עסקים קטנים ובינוניים	שליטה על שעות העבודה ועל עלות השכר, בלי תוכנה יקרה ובלי ציוד מיוחד
ראשי צוותים	מקום אחד לראות מי עובד, מי חסר, ולאשר תיקונים ואישורי מחלה
עובדים — במשרד ומהבית	דיווח פשוט מכל מחשב או טלפון, ושקיפות מלאה על השעות והשכר שלהם

המערכת מתאימה במיוחד לצוותים בעבודה **היברידית**, שבהם חלק מהימים עובדים מהמשרד וחלק מהבית.

💼 איך זה מועיל לבעלי עסקים

בעיה בעסק	הפתרון ב־CheckIn
לא יודעים מי עובד עכשיו, במיוחד מהבית	מסך **"מי במשמרת"** מתעדכן אוטומטית ומראה מי נכנס, מאיפה ומתי
עובדים שוכחים לדווח, ומגלים רק בסוף החודש	ימים עם **חסר בדיווח מסומנים באדום** — גם לעובד וגם לראש הצוות
תיקוני שעות "בדיעבד" בלי בקרה	**מכסת תיקונים שבועית** — מעבר לה, התיקון הופך לבקשה שראש הצוות צריך לאשר. אי אפשר לשלוח שתי בקשות לאותו יום
היעדרות בגלל מחלה בלי מסמך	העובד **מעלה אישור רפואי** עם הסבר, וראש הצוות **צופה במסמך ומאשר**
חישוב שכר ידני, טעויות ומחלוקות	**שכר מחושב אוטומטית** מהשעות, כולל שעות נוספות ב־125% ובונוס לראש הצוות
שינוי שכר באמצע חודש יוצר בלבול	**שינוי שכר נכנס לתוקף מתחילת החודש הבא**, וראש הצוות רואה מראש את השכר העתידי ומאיזה חודש הוא חל
עובד מאחר ולא ברור למה	**הערה לדיווח חריג** בכל כניסה, שמופיעה לראש הצוות בדוח

בשורה התחתונה: פחות זמן על ניהול שעות, פחות טעויות בתלוש, ושקיפות מלאה בין העובד למנהל.

🧭 מה כל משתמש יכול לעשות

עובד

- **דף הבית** – כניסה מהמשרד או מהבית (עם הערה לדיווח חריג), הפסקה, וסיום יום עם סיכום יומי. שעון חי ורשימת זמני העבודה של היום
- **הנוכחות שלי** – טבלה חודשית עם ניווט בין חודשים, סה"כ שעות וימים חסרים (מסומנים באדום)
  - **תיקון שעות** ליום חסר או ליום בלי יציאה. שני תיקונים בשבוע נכנסים מיד, ומעבר לזה התיקון נשלח לאישור ראש הצוות
  - **אישור מחלה** ליום חסר – כתיבת סיבת ההיעדרות והעלאת מסמך מהרופא (PDF או תמונה)
  - **הבקשות שלי** – כל בקשות התיקון של החודש והמצב שלהן: ממתין, מאושר או נדחה
- **השכר שלי** – שעות רגילות, שעות נוספות וסכום לתשלום

ראש צוות – כל מה שעובד יכול, ובנוסף:

- **ניהול עובדים** – רשימת עובדי הצוות, קוד הצטרפות לעובדים חדשים, עדכון שכר שעתי ותצוגת שכר עתידי
- **מי במשמרת** – מצב הצוות כרגע, עם רענון אוטומטי
- **דוח נוכחות** – חיפוש לפי שם, סינון לפי תאריכים ומיקום, מיון, והערות העובדים לכל כניסה
- **בקשות לתיקון שעות** – הבקשות של עובדים שעברו את המכסה, עם אישור או דחייה
- **אישורי מחלה** – צפייה במסמך ובהסבר של העובד, עם אישור או דחייה. יום מחלה שאושר מסומן "חולה" ולא נחשב חיסור

✅ עמידה בדרישות הפרויקט

דרישות המערכת

דרישה	איך זה מתקיים במערכת	איפה רואים את זה
כניסה למערכת	הרשמה והתחברות עם טוקן, שמירת המשתמש והתנתקות	מסך **התחברות**, מסך **הרשמה**, כפתור **התנתקות** בתפריט
שני סוגי משתמשים עם הרשאות שונות	עובד וראש צוות. כל מסך מוגן לפי תפקיד, והתפריט משתנה לפי התפקיד	**התפריט הצדדי** – לראש צוות יש 5 מסכים נוספים. עובד שמנסה להיכנס למסך של ראש צוות מועבר לדף הבית שלו
אובייקט מרכזי	**דיווח נוכחות**	**דף הבית** ← "זמני העבודה של היום"
לפחות 5 שדות משמעותיים	תאריך, שעת כניסה, שעת יציאה, מיקום, הערה, סיכום יומי, סוג סיום, דקות עבודה	**דוח נוכחות** (ראש צוות)
קטגוריה / סוג / סטטוס	**מיקום** (משרד / בית), **סטטוס יום** (נוכח, חסר, יציאה חסרה, חולה, חג ועוד) ו**סטטוס בקשה** (ממתין, מאושר, נדחה)	**הנוכחות שלי** ← עמודת סטטוס ו"הבקשות שלי" · **דף הבית** ← כפתורי כניסה

רשימת אובייקטים	טבלאות נוכחות, רשימת זמני העבודה של היום, רשימות בקשות ואישורים	**הנוכחות שלי**, **דוח נוכחות**, **בקשות לתיקון שעות**, **אישורי מחלה**
הוספה ועריכה	**הוספה:** כניסה לעבודה, השלמת יום חסר, העלאת אישור מחלה. **עריכה:** תיקון שעת יציאה, עדכון שכר עובד, אישור או דחייה של בקשות	**דף הבית** · **הנוכחות שלי** ← "תיקון" / "אישור מחלה" · **ניהול עובדים** ← "ערוך שכר"
חיפוש, סינון או מיון	**חיפוש** לפי שם, **סינון** לפי טווח תאריכים ומיקום, **מיון** לפי תאריך, שם, שעת כניסה או שעות, בסדר עולה או יורד	**דוח נוכחות** (ראש צוות)
מידע משרת ונשמר במסד נתונים	כל המידע מגיע מהשרת ונשמר ב־PostgreSQL, כולל קבצי אישורי המחלה	בכל המסכים — הנתונים נשמרים גם אחרי רענון ובכניסה ממחשב אחר

חובה ב־React

דרישה	איך זה מתקיים	איפה רואים את זה
React + TypeScript	כל הפרויקט ב־TypeScript, עם טיפוסים לכל מבנה נתונים שמגיע מהשרת	בכל הקוד
React Router	נתיב לכל מסך, מסכים מוגנים לפי תפקיד, הפניות אוטומטיות וניווט בתפריט	**התפריט הצדדי** · כתובת לא קיימת מחזירה לדף הבית
React Hook Form	טפסי התחברות, הרשמה, תיקון שעות ואישור מחלה (כולל העלאת קובץ). טופס ההרשמה משתנה לפי התפקיד שנבחר	מסך **התחברות**, מסך **הרשמה**, **הנוכחות שלי** ← "תיקון" ו"אישור מחלה"
React Query	טעינת כל הנתונים מהשרת דרך hook משותף. כשמשנים סינון או חודש, הנתונים נטענים מחדש אוטומטית, ואחרי אישור בקשה הרשימה מתרעננת	כל המסכים שמציגים נתונים
Redux Toolkit / Context	**Context** שמחזיק את המשתמש המחובר ואת הטוקן	**התפריט הצדדי** — שם המשתמש · שמירת ההתחברות אחרי רענון

חשוב להקפיד על

נושא	איך זה מתקיים	איפה רואים את זה
חלוקה נכונה לקומפוננטות	מסכים מלאים מופרדים מחלקים לשימוש חוזר, וכל חלק עם תפקיד אחד	**דוח נוכחות**: סינון, טבלה ודף שמחבר ביניהם · **אישורי מחלה**: דף ורכיב לאישור אחד
שימוש נכון ב־State וב־Props	הסינון נשמר בדף ועובר גם לרכיב הסינון וגם לטבלה. החודש שנבחר קובע גם את הטבלה וגם את רשימת הבקשות	**דוח נוכחות**, **הנוכחות שלי**
שימוש ב־Hooks	`useId` , `useEffect` , `useState` ו־hooks שכתבתי: `useAuth` ו־`useFetch`	**דף הבית** ← שעון חי · כל המסכים שטוענים נתונים
Routing בין עמודים	ניווט בין כל המסכים, הפניה אוטומטית לדף הבית לפי התפקיד, וחסימת מסכים לא מורשים	**התפריט הצדדי**

טפסים עם Validation	שדות חובה, אימייל, סיסמה של 8 תווים לפחות, שכר בין 0 ל־1000, חובת צירוף קובץ. הודעת שגיאה ליד כל שדה, ושגיאות מהשרת מוצגות בטופס	מסך **הרשמה** · **אישור מחלה** — לנסות לשלוח בלי קובץ
קריאות מסודרות לשרת	כל הקריאות מרוכזות במקום אחד, לפי תחום, והטוקן מתווסף לכל בקשה אוטומטית — גם בפתיחת מסמך	בכל המסכים
הצגת Loading	חיווי טעינה בזמן שהנתונים נטענים, וכפתורים מושבתים בזמן שליחה	**הנוכחות שלי**, **דוח נוכחות**, **בקשות לתיקון שעות**, **אישורי מחלה**
טיפול בשגיאות	הודעות שגיאה ברורות מהשרת (למשל בקשה כפולה לאותו יום, או קובץ לא נתמך), והתנתקות אוטומטית כשההתחברות פגה	מסך **התחברות** — לנסות סיסמה שגויה
מניעת שכפול קוד	רכיב קלט אחד לכל הטפסים, hook אחד לטעינת נתונים, פונקציות עזר משותפות, ופונקציה אחת לכפתורי אישור ודחייה	כל שדות הטפסים באתר נראים ומתנהגים אותו דבר
בלי קומפוננטה אחת גדולה	המערכת מחולקת ל־11 מסכים ול־27 קומפוננטות קטנות	—

הגשה

דרישה	מצב
קוד בגיט, בלי `node_modules`	✓ שני ריפואים: לקוח ושרת
העלאה לענן: שרת, לקוח ומסד נתונים	✓ **לקוח** — Render · **שרת** — Render · **מסד נתונים** — PostgreSQL ב־Neon

🛠️ טכנולוגיות

לקוח: Bootstrap · Axios · TanStack React Query · React Hook Form · React Router · Vite · TypeScript · React 19 · CSS · (RTL) 5 מותאם לטלפון

שרת: Swagger · Zod · JWT · PostgreSQL · Prisma · TypeScript · Express · Node.js

💻 הרצה מקומית

השרת

1. `npm install`
2. `npm run db:local` — מסד נתונים מקומי (להשאיר פתוח)
3. `npm run dev` — השרת, בפורט 3000

הלקוח

1. `npm install`
2. ליצור קובץ `.env` עם השורה `VITE_API_URL=http://localhost:3000`
3. `npm run dev` — האתר, בכתובת `http://localhost:5180`